

Javascript_Day1_week2_Handson_Gurunath_Rageni

Problem 1: Personal Notes Saver using LocalStorage (Level-1)

Scenario

You are building a simple web page where users can write daily notes and save them in their browser without using a server.

Requirements

- A textarea for writing notes.
- A Save button (using onclick).
- A Clear button.
- Notes must:

Be stored in localStorage

Automatically load when the page refreshes

- Display stored note on page load.

Technical Constraints

- Must use:
 - onclick inline event
 - localStorage.setItem()
 - localStorage.getItem()
 - localStorage.removeItem()
- No backend/database.
- Pure HTML + JavaScript only.
- Data stored as key-value pair.

Code:

<!--Problem 1: Greeting Generator (Level-1)

Scenario

A simple webpage should greet users based on the name they enter. This helps beginners understand how JavaScript functions accept parameters and how variables behave inside functions.

Requirements

- Create an input box to enter a user's name.
- Create a button labeled "Generate Greeting".
- When the button is clicked:
 - A JavaScript function should accept the name as a parameter.
 - Display a greeting message like:
"Hello, Rahul! Welcome to our website."
 - Display the greeting inside a <p> element.-->

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Personal Notes Saver</title>
```

```
</head>
```

```
<body>
```

```
  <h2>My Daily Notes</h2>
```

```
  <textarea id="noteArea" rows="10" cols="40" placeholder="Write your notes here..."></textarea>
```

```
  <br><br>
```

```
  <!-- Buttons with inline onclick -->
```

```
  <button onclick="saveNote()">Save</button>
```

```
  <button onclick="clearNote()">Clear</button>
```

```
<script>
```

```
  // Load note automatically when page loads
```

```
  window.onload = function() {
```

```
    let savedNote = localStorage.getItem("myNote");
```

```
    if (savedNote !== null) {
```

```
        document.getElementById("noteArea").value = savedNote;
    }
};

// Save note to localStorage
function saveNote() {
    let note = document.getElementById("noteArea").value;
    localStorage.setItem("myNote", note);
    alert("Note saved successfully!");
}

// Clear note from textarea and localStorage
function clearNote() {
    localStorage.removeItem("myNote");
    document.getElementById("noteArea").value = "";
    alert("Note cleared!");
}

</script>

</body>

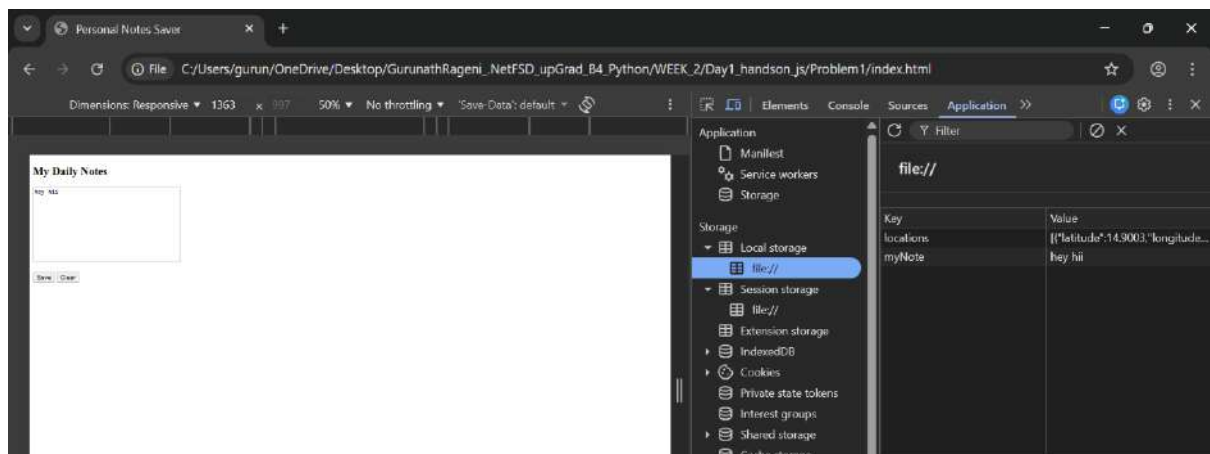
</html>
```

Code Screenshots:

```
WEEK_2 > Day1_handson_js > Problem1 > index.html > html > head
1  <!--Problem 1: Greeting Generator (Level-1)
2  Scenario
3  A simple webpage should greet users based on the name they enter. This helps beginners unde
4  ★ Requirements
5  @Create an input box to enter a user's name.
6  @Create a button labeled "Generate Greeting".
7  @When the button is clicked:
8  @A JavaScript function should accept the name as a parameter.
9  @Display a greeting message like:
10 "Hello, Rahul! Welcome to our website."
11 @Display the greeting inside a <p> element.-->
12
13
14 <!DOCTYPE html>
15 <html>
16 <head>
17   <title>Personal Notes Saver</title>
18 </head>
19 <body>
20
21   <h2>My Daily Notes</h2>
22
23
24   <textarea id="noteArea" rows="10" cols="40" placeholder="Write your notes here..."></te
25   <br><br>
26
27   <!-- Buttons with inline onclick -->
28   <button onclick="saveNote()">Save</button>
29   <button onclick="clearNote()">Clear</button>
```

```
30   <script>
31     // Load note automatically when page loads
32     window.onload = function() {
33       let savedNote = localStorage.getItem("myNote");
34       if (savedNote !== null) {
35         document.getElementById("noteArea").value = savedNote;
36       }
37     };
38
39     // Save note to localStorage
40     function saveNote() {
41       let note = document.getElementById("noteArea").value;
42       localStorage.setItem("myNote", note);
43       alert("Note saved successfully!");
44     }
45
46     // Clear note from textarea and localStorage
47     function clearNote() {
48       localStorage.removeItem("myNote");
49       document.getElementById("noteArea").value = "";
50       alert("Note cleared!");
51     }
52   </script>
53 </body>
54 </html>
```

Output Screenshot:



Explanation:

This webpage demonstrates client-side data persistence using the Web Storage API (localStorage) along with dynamic DOM manipulation. When the page loads, the `window.onload` event retrieves previously saved data using `localStorage.getItem()` and automatically populates the textarea, ensuring data persistence across page refreshes. Inline onclick event handlers trigger JavaScript functions that use `setItem()` to store notes and `removeItem()` to delete them from browser storage. Overall, the implementation showcases basic state management and storage handling entirely on the client side without requiring a server or database.

Problem 2: Live Form Validation with Events (Level-1)

Scenario

Create a simple registration form that validates user input when fields change.

📌 Requirements

- Fields:

Name

Email

Age

- Use:

onchange

onclick

- Validate:

Name cannot be empty

Email must contain "@"

Age must be greater than 18

- Display validation message dynamically.
- Store valid user data in sessionStorage.

Technical Constraints

- Must use inline onchange events.
- Store valid data using:

sessionStorage.setItem()

- No external libraries.
- Use basic JavaScript only.

Learning Outcome

You will be able to:

- onchange event usage
- Form validation logic
- Difference between localStorage and sessionStorage
- Dynamic DOM updates

Problem 3: Location-Based Weather Logger (Level-2)

Scenario

Create a web application that fetches the user's geographic location and stores location history in localStorage.

Requirements

- Button: **Get My Location**
- Use:

navigator.geolocation.getCurrentPosition()

- Display:

Latitude

Longitude

- Handle:

Permission denied

Timeout

Location unavailable

- Save last 5 location entries in localStorage.
- Display location history on page load.

Technical Constraints

- Must handle:

Success callback

Error callback

- Use browser permission handling.
- Store data as JSON using:

JSON.stringify()

JSON.parse()

- Use inline event (onclick).

Code:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Live Form Validation</title>
```

```
</head>
```

```
<body>
```

```
  <h2>Registration Form</h2>
```

```
  <!-- Name Field -->
```

```
  <label>Name:</label>
```

```
  <input type="text" id="name" onchange="validateName()">
```

```
  <p id="nameError" style="color:red;"></p>
```

```
<br><br>
```

```
<!-- Email Field -->
```

```
<label>Email:</label>
```

```
<input type="text" id="email" onchange="validateEmail()">
```

```
<p id="emailError" style="color:red;"></p>
```

```
<br><br>
```

```
<!-- Age Field -->
```

```
<label>Age:</label>
```

```
<input type="number" id="age" onchange="validateAge()">
```

```
<p id="ageError" style="color:red;"></p>
```

```
<br><br>
```

```
<!-- Submit Button -->
```

```
<button onclick="saveData()">Register</button>
```

```
<p id="successMessage" style="color:green;"></p>
```

```
<script>
```

```
function validateName() {
```

```
    let name = document.getElementById("name").value;
```

```
    if (name.trim() === "") {
```

```
        document.getElementById("nameError").innerText = "Name cannot be empty";
```

```
        return false;
```

```
    } else {
```

```
        document.getElementById("nameError").innerText = "";
```

```
        return true;
    }
}
```

```
function validateEmail() {
    let email = document.getElementById("email").value;

    if (!email.includes("@")) {
        document.getElementById("emailError").innerText = "Email must contain @";
        return false;
    } else {
        document.getElementById("emailError").innerText = "";
        return true;
    }
}
```

```
function validateAge() {
    let age = document.getElementById("age").value;

    if (age <= 18) {
        document.getElementById("ageError").innerText = "Age must be greater than 18";
        return false;
    } else {
        document.getElementById("ageError").innerText = "";
        return true;
    }
}
```

```
function saveData() {

    let isNameValid = validateName();
```

```
let isEmailValid = validateEmail();

let isAgeValid = validateAge();


if (isNameValid && isEmailValid && isAgeValid) {

    let user = {
        name: document.getElementById("name").value,
        email: document.getElementById("email").value,
        age: document.getElementById("age").value
    };

    sessionStorage.setItem("userData", JSON.stringify(user));

    document.getElementById("successMessage").innerText =
        "Registration Successful! Data stored in sessionStorage.";
} else {
    document.getElementById("successMessage").innerText = "";
}
}

</script>

</body>

</html>
```

Code Screenshots:

WEEK_2 > Day1_handson.js > Problem2 > index.html > html > body > input#age

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Live Form Validation</title>
5  </head>
6  <body>
7
8      <h2>Registration Form</h2>
9
10     <!-- Name Field -->
11     <label>Name:</label>
12     <input type="text" id="name" onchange="validateName()">
13     <p id="nameError" style="color: red;"></p>
14
15     <br><br>
16
17     <!-- Email Field -->
18     <label>Email:</label>
19     <input type="text" id="email" onchange="validateEmail()">
20     <p id="emailError" style="color: red;"></p>
21
22     <br><br>
23
24     <!-- Age Field -->
25     <label>Age:</label>
26     <input type="number" id="age" onchange="validateAge()">
27     <p id="ageError" style="color: red;"></p>
28
29     <br><br>
```

```

29     <br><br>
30
31     <!-- Submit Button -->
32     <button onclick="saveData()">Register</button>
33
34     <p id="successMessage" style="color: green;"></p>
35
36 </script>
37
38 function validateName() {
39     let name = document.getElementById("name").value;
40
41     if (name.trim() === "") {
42         document.getElementById("nameError").innerText = "Name cannot be empty";
43         return false;
44     } else {
45         document.getElementById("nameError").innerText = "";
46         return true;
47     }
48 }
49
50 function validateEmail() {
51     let email = document.getElementById("email").value;
52
53     if (!email.includes("@")) {
54         document.getElementById("emailError").innerText = "Email must contain @";
55         return false;

```

```

56         document.getElementById("emailError").innerText = "Email must contain @";
57         return false;
58     } else {
59         document.getElementById("emailError").innerText = "";
60         return true;
61     }
62 }
63
64 function validateAge() {
65     let age = document.getElementById("age").value;
66
67     if (age <= 18) {
68         document.getElementById("ageError").innerText = "Age must be greater than 18";
69         return false;
70     } else {
71         document.getElementById("ageError").innerText = "";
72         return true;
73     }
74 }
75
76 function saveData() {
77     let isNameValid = validateName();
78     let isEmailValid = validateEmail();
79     let isAgeValid = validateAge();

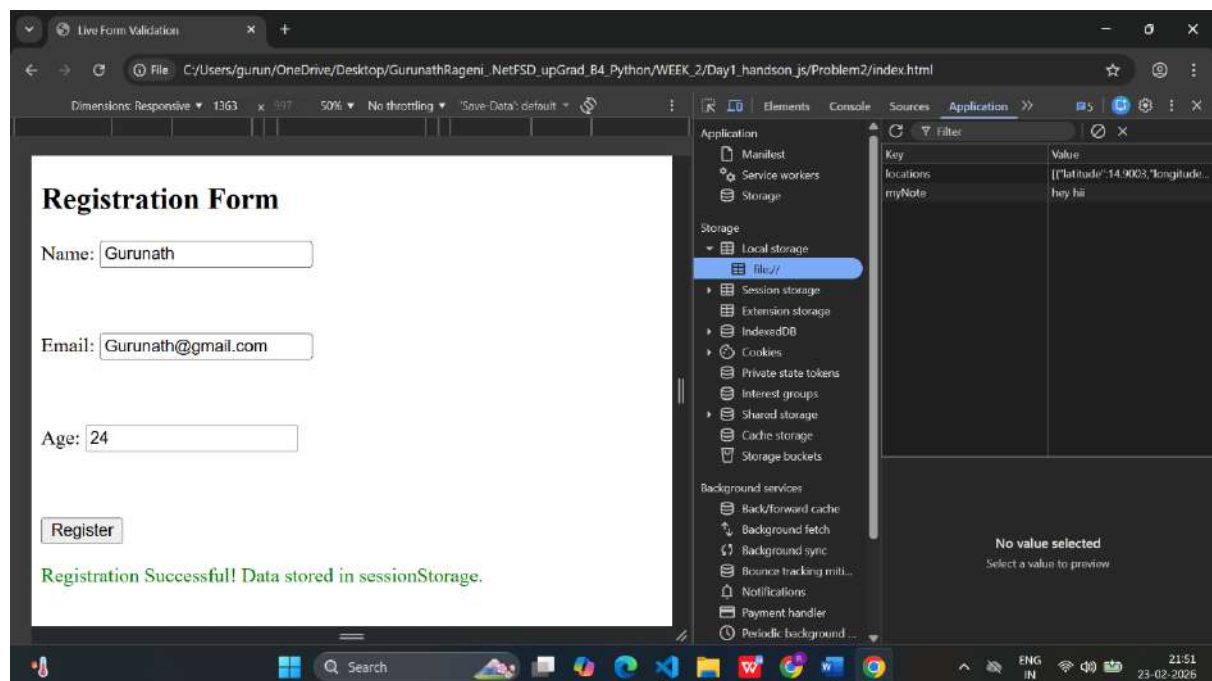
```

```

78     let isAgeValid = validateAge();
79
80     if (isNameValid && isEmailValid && isAgeValid) {
81
82         let user = {
83             name: document.getElementById("name").value,
84             email: document.getElementById("email").value,
85             age: document.getElementById("age").value
86         };
87
88         sessionStorage.setItem("userData", JSON.stringify(user));
89
90         document.getElementById("successMessage").innerText =
91             "Registration Successful! Data stored in sessionStorage.";
92     } else {
93         document.getElementById("successMessage").innerText = "";
94     }
95 }
96
97 </script>
98
99 </body>
100 </html>

```

Output Screenshot:



Explanation:

This webpage implements real-time form validation using inline onchange and onclick events to validate user inputs dynamically. Valid data is stored in sessionStorage as a JSON object, demonstrating client-side validation and temporary browser-based data persistence.

Problem 3: Location-Based Weather Logger (Level-2)

Scenario

Create a web application that fetches the user's geographic location and stores location history in localStorage.

📌 Requirements

- Button: **Get My Location**
- Use:

navigator.geolocation.getCurrentPosition()

- Display:

Latitude

Longitude

- Handle:

Permission denied

Timeout

Location unavailable

- Save last 5 location entries in localStorage.
- Display location history on page load.

🔧 Technical Constraints

- Must handle:

Success callback

Error callback

- Use browser permission handling.
- Store data as JSON using:

JSON.stringify()

JSON.parse()

- Use inline event (onclick).

Code:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>

  <title>Location-Based Weather Logger</title>
</head>

<body>

  <h2>Location Logger</h2>

  <button onclick="getLocation()">Get My Location</button>

  <h3>Current Location</h3>
  <p id="output"></p>
  <p id="error" style="color:red;"></p>

  <h3>Last 5 Locations</h3>
  <ul id="historyList"></ul>

<script>

// Load history when page loads
window.onload = function () {
  displayHistory();
};

function getLocation() {

  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(
      successCallback,
      errorCallback,
      { timeout: 10000 }
    );
```

```
    } else {  
        document.getElementById("error").innerText =  
            "Geolocation is not supported by this browser.";  
    }  
}
```

```
// SUCCESS CALLBACK
```

```
function successCallback(position) {  
  
    let latitude = position.coords.latitude;  
    let longitude = position.coords.longitude;  
  
    document.getElementById("output").innerText =  
        "Latitude: " + latitude +  
        " | Longitude: " + longitude;  
  
    document.getElementById("error").innerText = "";  
  
    let locationData = {  
        latitude: latitude,  
        longitude: longitude,  
        time: new Date().toLocaleString()  
    };  
  
    saveLocation(locationData);  
}
```

```
// ERROR CALLBACK
```

```
function errorCallback(error) {  
  
    let message = "";
```

```
switch (error.code) {  
  case error.PERMISSION_DENIED:  
    message = "Permission Denied by user.";  
    break;  
  case error.TIMEOUT:  
    message = "Location request timed out.";  
    break;  
  case error.POSITION_UNAVAILABLE:  
    message = "Location information unavailable.";  
    break;  
  default:  
    message = "Unknown error occurred.";  
}  
  
document.getElementById("error").innerText = message;  
}  
  
// Save last 5 locations  
function saveLocation(newLocation) {  
  
  let locations = JSON.parse(localStorage.getItem("locations")) || [];  
  
  locations.unshift(newLocation); // Add newest at top  
  
  if (locations.length > 5) {  
    locations.pop(); // Remove oldest  
  }  
  
  localStorage.setItem("locations", JSON.stringify(locations));  
}
```

```
    displayHistory();
}
// Display history
function displayHistory() {

    let locations = JSON.parse(localStorage.getItem("locations")) || [];

    let list = document.getElementById("historyList");
    list.innerHTML = "";

    locations.forEach(function (loc) {
        let li = document.createElement("li");
        li.innerText =
            "Lat: " + loc.latitude +
            " | Lon: " + loc.longitude +
            " | Time: " + loc.time;
        list.appendChild(li);
    });
}
</script>
</body>
</html>
```

Code Screenshots:

WEEK_2 > Day1_handson_js > Problem3 > <> index.html > html > body > script > onload

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Location-Based Weather Logger</title>
5  </head>
6  <body>
7  |
8  |   <h2>Location Logger</h2>
9  |
10 |   <button onclick="getLocation()">Get My Location</button>
11 |
12 |   <h3>Current Location</h3>
13 |   <p id="output"></p>
14 |   <p id="error" style="color: red;"></p>
15 |
16 |   <h3>Last 5 Locations</h3>
17 |   <ul id="historyList"></ul>
18 |
19 </script>
20
21 // Load history when page loads
22 window.onload = function () {
23 |   displayHistory();
24 | };
25
26 function getLocation() {
27 |
28 |   if (navigator.geolocation) {
29 |     navigator.geolocation.getCurrentPosition(
```

```
29 |     navigator.geolocation.getCurrentPosition(
30 |       successCallback,
31 |       errorCallback,
32 |       { timeout: 10000 }
33 |     );
34 |   } else {
35 |     document.getElementById("error").innerText =
36 |       "Geolocation is not supported by this browser.";
37 |   }
38 | }
39
40 // SUCCESS CALLBACK
41 function successCallback(position) {
42 |
43 |   let latitude = position.coords.latitude;
44 |   let longitude = position.coords.longitude;
45 |
46 |   document.getElementById("output").innerText =
47 |     "Latitude: " + latitude +
48 |     " | Longitude: " + longitude;
49 |
50 |   document.getElementById("error").innerText = "";
51 |
52 |   let locationData = {
```

```
41 function successCallback(position) {
42     document.getElementById("output").innerText =
43         "Latitude: " + latitude +
44         " | Longitude: " + longitude;
45
46     document.getElementById("error").innerText = "";
47
48     let locationData = {
49         latitude: latitude,
50         longitude: longitude,
51         time: new Date().toLocaleString()
52     };
53
54     saveLocation(locationData);
55 }
56
57 // ERROR CALLBACK
58 function errorCallback(error) {
59
60     let message = "";
61
62     switch (error.code) {
63         case error.PERMISSION_DENIED:
64             message = "Permission Denied by user.";
65             break;
66         case error.TIMEOUT:
```

```

70     case error.TIMEOUT:
71         message = "Location request timed out.";
72         break;
73     case error.POSITION_UNAVAILABLE:
74         message = "Location information unavailable.";
75         break;
76     default:
77         message = "Unknown error occurred.";
78 }
79
80 document.getElementById("error").innerText = message;
81 }
82
83 // Save last 5 locations
84 function saveLocation(newLocation) {
85
86     let locations = JSON.parse(localStorage.getItem("locations")) || [];
87
88     locations.unshift(newLocation); // Add newest at top
89
90     if (locations.length > 5) {
91         locations.pop(); // Remove oldest
92     }
93
94     localStorage.setItem("locations", JSON.stringify(locations));

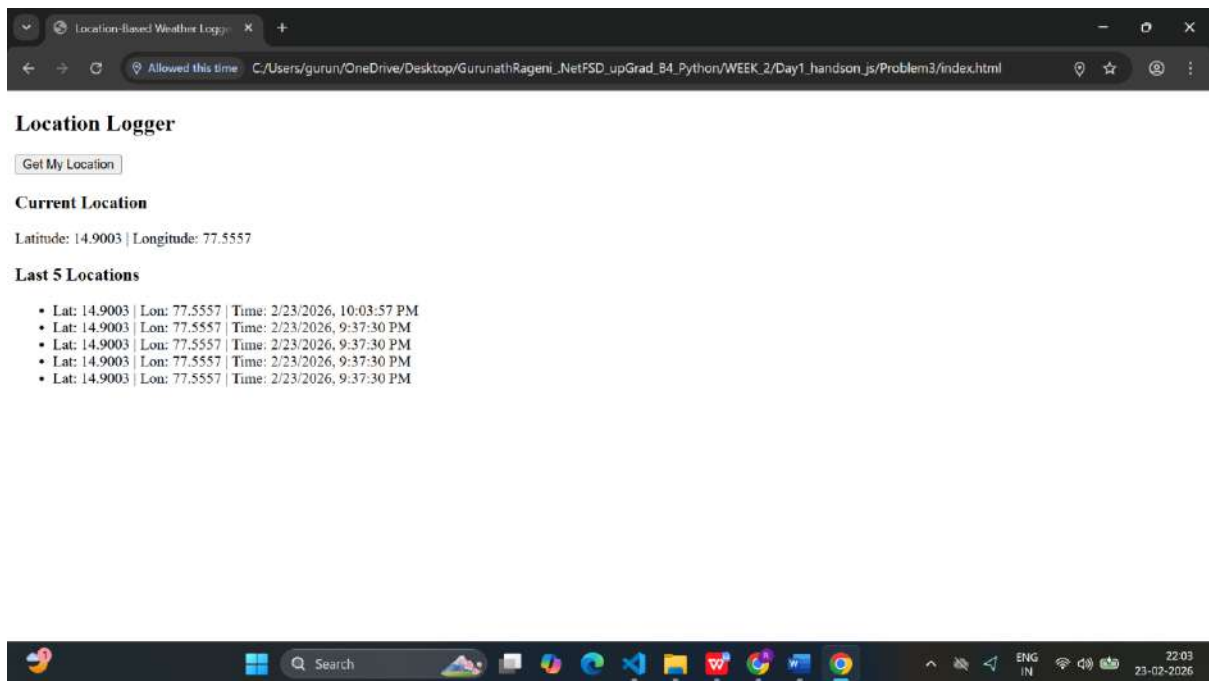
```

```

93
94     localStorage.setItem("locations", JSON.stringify(locations));
95
96     displayHistory();
97 }
98 // Display history
99 function displayHistory() {
100
101     let locations = JSON.parse(localStorage.getItem("locations")) || [];
102
103     let list = document.getElementById("historyList");
104     list.innerHTML = "";
105
106     locations.forEach(function (loc) {
107         let li = document.createElement("li");
108         li.innerText =
109             "Lat: " + loc.latitude +
110             " | Lon: " + loc.longitude +
111             " | Time: " + loc.time;
112         list.appendChild(li);
113     });
114 }
115 </script>
116 </body>
117 </html>

```

Code Output:



Explanation:

This application uses the **Geolocation API** (`navigator.geolocation.getCurrentPosition`) to fetch the user's latitude and longitude with proper success and error callback handling. It stores the last five location entries in `localStorage` using `JSON.stringify()` and `JSON.parse()`, ensuring persistent structured data management and dynamic history rendering on page load.

Problem 4: Mini Expense Tracker using Client-Side Database (Level-2)

Scenario

Develop a client-side expense tracker where users can add, view, and delete expenses using a browser-supported database.

📌 Requirements

1. Fields:
 - Expense Title
 - Amount
 - Date
2. Buttons:
 - Add Expense
 - View Expenses
 - Delete Expense

3. Use:
 - Client-side database (Web SQL or IndexedDB)
4. Execute SQL-like queries:
 - CREATE TABLE
 - INSERT
 - SELECT
 - DELETE
5. Maintain transaction handling.
6. Display expense list dynamically.

Technical Constraints

- Must use:
 - Client-side DB transactions
 - SQL execution methods
- Must handle:
 - Transaction errors
 - Query errors
- No server/database allowed.
- Pure JavaScript + HTML.

Code:

```
<!DOCTYPE html>

<html>

<head>

  <title>Mini Expense Tracker</title>

</head>

<body>


<h2>Mini Expense Tracker</h2>


<label>Title:</label>

<input type="text" id="title"><br><br>
```

```
<label>Amount:</label>
```

```
<input type="number" id="amount"><br><br>
```

```
<label>Date:</label>
```

```
<input type="date" id="date"><br><br>
```

```
<button onclick="addExpense()">Add Expense</button>
```

```
<button onclick="viewExpenses()">View Expenses</button>
```

```
<h3>Expense List</h3>
```

```
<ul id="expenseList"></ul>
```

```
<p id="error" style="color:red;"></p>
```

```
<script>
```

```
let db;
```

```
// CREATE DATABASE + TABLE
```

```
let request = indexedDB.open("ExpenseDB", 1);
```

```
request.onupgradeneeded = function(event) {
```

```
    db = event.target.result;
```

```
    // Simulates: CREATE TABLE expenses
```

```
    let objectStore = db.createObjectStore("expenses", {
```

```
        keyPath: "id",
```

```
        autoIncrement: true
```

```
    });
```

```
    objectStore.createIndex("title", "title", { unique: false });
```

```
};
```

```
request.onsuccess = function(event) {  
    db = event.target.result;  
    console.log("Database opened successfully");  
};
```

```
request.onerror = function(event) {  
    document.getElementById("error").innerText =  
        "Database error: " + event.target.errorCode;  
};
```

```
// INSERT → add()
```

```
function addExpense() {
```

```
    let title = document.getElementById("title").value;  
    let amount = document.getElementById("amount").value;  
    let date = document.getElementById("date").value;
```

```
    if (!title || !amount || !date) {  
        alert("All fields required");  
        return;  
    }
```

```
    let transaction = db.transaction(["expenses"], "readwrite");
```

```
    transaction.onerror = function() {  
        document.getElementById("error").innerText =  
            "Transaction failed";  
    };  
};
```

```

let store = transaction.objectStore("expenses");

let expense = {
  title: title,
  amount: amount,
  date: date
};

let addRequest = store.add(expense);

addRequest.onsuccess = function() {
  alert("Expense Added Successfully");
  viewExpenses();
};

addRequest.onerror = function() {
  document.getElementById("error").innerText =
    "Error adding expense";
};
}

// SELECT → getAll()
function viewExpenses() {

  let transaction = db.transaction(["expenses"], "readonly");
  let store = transaction.objectStore("expenses");

  let request = store.getAll();

  request.onsuccess = function(event) {

```

```

let expenses = event.target.result;

let list = document.getElementById("expenseList");
list.innerHTML = "";

expenses.forEach(function(exp) {

    let li = document.createElement("li");
    li.innerHTML =
        exp.title + " | ₹" + exp.amount + " | " + exp.date +
        " <button onclick='deleteExpense(" + exp.id + ")'>Delete</button>";

    list.appendChild(li);
});
};

request.onerror = function() {
    document.getElementById("error").innerText =
        "Error fetching expenses";
};
}

// DELETE → delete()
function deleteExpense(id) {

    let transaction = db.transaction(["expenses"], "readwrite");
    let store = transaction.objectStore("expenses");

    let request = store.delete(id);

    request.onsuccess = function() {

```

```
        alert("Expense Deleted");  
        viewExpenses();  
    };  
  
    request.onerror = function() {  
        document.getElementById("error").innerText =  
            "Error deleting expense";  
    };  
}  
  
</script>  
  
</body>  
</html>
```

Code Screenshots:

WEEK_2 > Day1_handson_js > Problem4 > index.html > html > body > button

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>Mini Expense Tracker</title>
5  </head>
6  <body>
7
8  <h2>Mini Expense Tracker</h2>
9
10 <label>Title:</label>
11 <input type="text" id="title"><br><br>
12
13 <label>Amount:</label>
14 <input type="number" id="amount"><br><br>
15
16 <label>Date:</label>
17 <input type="date" id="date"><br><br>
18
19 <button onclick="addExpense()">Add Expense</button>
20 <button onclick="viewExpenses()">View Expenses</button>
21
22 <h3>Expense List</h3>
23 <ul id="expenseList"></ul>
24
25 <p id="error" style="color: red;"></p>
26
27 <script>
28
29 let db;
```

```
29 let db;
30
31 // CREATE DATABASE + TABLE
32 let request = indexedDB.open("ExpenseDB", 1);
33
34 request.onupgradeneeded = function(event) {
35     db = event.target.result;
36
37     // Simulates: CREATE TABLE expenses
38     let objectStore = db.createObjectStore("expenses", {
39         keyPath: "id",
40         autoIncrement: true
41     });
42
43     objectStore.createIndex("title", "title", { unique: false });
44 };
45
46 request.onsuccess = function(event) {
47     db = event.target.result;
48     console.log("Database opened successfully");
49 };
50
51 request.onerror = function(event) {
52     document.getElementById("error").innerText =
53         "Database error: " + event.target.errorCode;
54 };
```

```

55
56 // INSERT → add()
57 function addExpense() {
58
59     let title = document.getElementById("title").value;
60     let amount = document.getElementById("amount").value;
61     let date = document.getElementById("date").value;
62
63     if (!title || !amount || !date) {
64         alert("All fields required");
65         return;
66     }
67
68     let transaction = db.transaction(["expenses"], "readwrite");
69
70     transaction.onerror = function() {
71         document.getElementById("error").innerText =
72             "Transaction failed";
73     };
74
75     let store = transaction.objectStore("expenses");
76
77     let expense = {
78         title: title,
79         amount: amount,
80         date: date

```

```

83     let addRequest = store.add(expense);
84
85     addRequest.onsuccess = function() {
86         alert("Expense Added Successfully");
87         viewExpenses();
88     };
89
90     addRequest.onerror = function() {
91         document.getElementById("error").innerText =
92             "Error adding expense";
93     };
94 }
95
96 // SELECT → getAll()
97 function viewExpenses() {
98
99     let transaction = db.transaction(["expenses"], "readonly");
100     let store = transaction.objectStore("expenses");
101
102     let request = store.getAll();
103
104     request.onsuccess = function(event) {
105
106         let expenses = event.target.result;

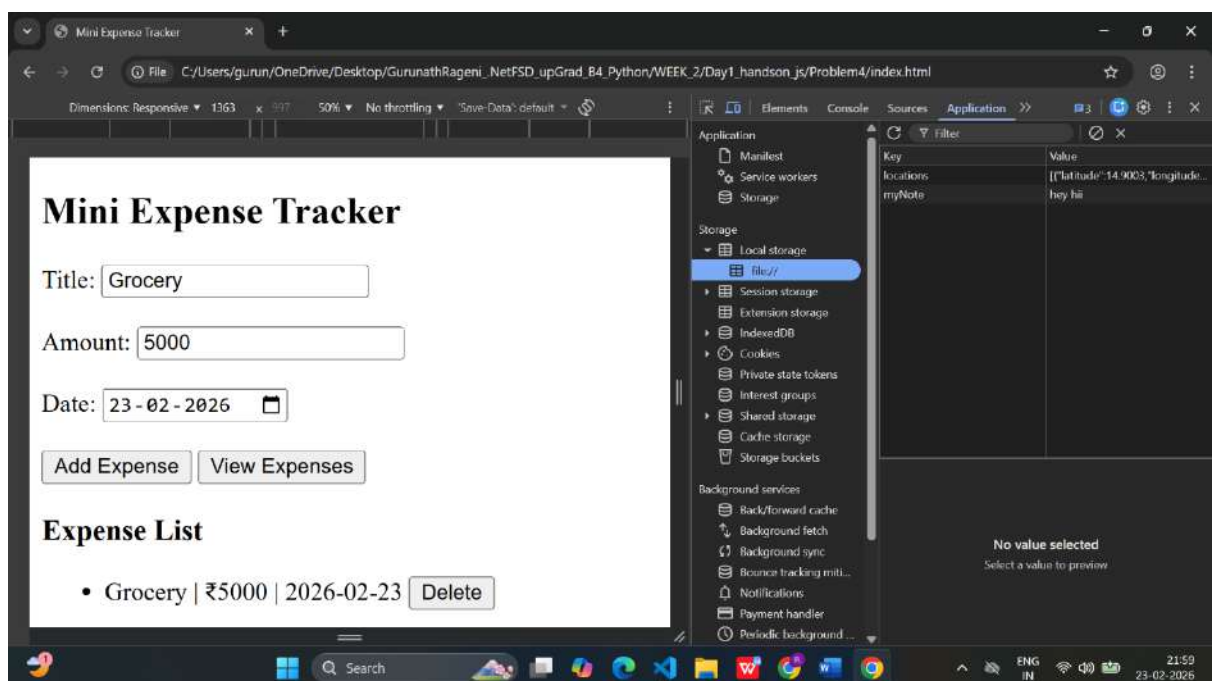
```

```

127
128 // DELETE → delete()
129 function deleteExpense(id) {
130
131     let transaction = db.transaction(["expenses"], "readwrite");
132     let store = transaction.objectStore("expenses");
133
134     let request = store.delete(id);
135
136     request.onsuccess = function() {
137         alert("Expense Deleted");
138         viewExpenses();
139     };
140
141     request.onerror = function() {
142         document.getElementById("error").innerText =
143             "Error deleting expense";
144     };
145 }
146
147 </script>
148
149 </body>
150 </html>

```

Output Screenshot:



Explanation:

This Mini Expense Tracker uses **IndexedDB** as a client-side database to store structured expense data persistently within the browser. It simulates SQL-like operations using `objectStore` (`CREATE TABLE`), `add()` (`INSERT`), `getAll()` (`SELECT`), and `delete()` (`DELETE`) inside transactions, while dynamically rendering the expense list in the DOM with proper error and transaction handling.